DS H.W SET 6

1.Simulate the undo and redo option in MS-Word packages using a doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node
{
   char action[50];
   struct Node *prev;
   struct Node *next;
};
struct Node *head = NULL;
struct Node *current = NULL;
void addAction(char action[])
{
   struct Node *newNode =
      (struct Node *)malloc(sizeof(struct Node));
   strcpy(newNode->action, action);
   newNode->next = NULL;
   newNode->prev = NULL;
   if(head == NULL)
   {
      head = newNode;
      current = newNode;
      return;
   }
   if(current->next != NULL)
   {
      struct Node *temp = current->next;
      while(temp != NULL)
      {
         struct Node *del = temp;
         temp = temp->next;
```

```
        free(del);
      }
      current->next = NULL;
   }
   newNode->prev = current;
   current->next = newNode;
   current = newNode;
}
void undo()
{
   if(current == NULL)
   {
      printf("No actions available.\n");
      return;
   }
   if(current->prev == NULL)
   {
      printf("Cannot Undo further.\n");
      return;
   }
   current = current->prev;
   printf("Undo Successful.\n");
}
```

Output

```
----- UNDO / REDO MENU -----
1. Add Action
2. Undo
3. Redo
4. Display Current Action
5. Display History
6. Exit
Enter your choice: 1
Enter action: Type A
```

Enter your choice: 1

Enter action: Type B

Enter your choice: 1

Enter action: Type C

Enter your choice: 5

Action History:

Type A

Type B

[Type C] <-- Current

2.Given two sorted lists L1 and L2 write a procedure to compute (L1 intersection L2) and (L1 union L2) using basic list operations.

```
#include <stdio.h>

#include <stdlib.h>
// Structure for Linked List Node
struct Node
{
   int data;
   struct Node *next;
};
// Create New Node
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
// Insert at End
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
}
// Display Linked List
void display(struct Node *head)
{
   while(head != NULL)
   {
      printf("%d", head->data);
      if(head->next != NULL)
         printf(" -> ");
      head = head->next;
   }
   printf(" -> NULL\n");
}
// Find Union
struct Node* unionList(struct Node *l1, struct Node *l2)
{
```

```
   struct Node *result = NULL;
   while(l1 != NULL && l2 != NULL)
   {
      if(l1->data < l2->data)
      {
         insert(&result, l1->data);
         l1 = l1->next;
      }
      else if(l1->data > l2->data)
      {
         insert(&result, l2->data);
         l2 = l2->next;
      }
      else
      {
         insert(&result, l1->data);
         l1 = l1->next;
         l2 = l2->next;
      }
   }
   while(l1 != NULL)
   {
      insert(&result, l1->data);
      l1 = l1->next;
   }
   while(l2 != NULL)
   {
      insert(&result, l2->data);
      l2 = l2->next;
   }
   return result;
}
// Find Intersection
struct Node* intersectionList(struct Node *l1, struct Node *l2)
{
   struct Node *result = NULL;
   while(l1 != NULL && l2 != NULL)
   {
      if(l1->data < l2->data)
      {
         l1 = l1->next;
      }
      else if(l1->data > l2->data)
      {
         l2 = l2->next;
      }
      else
      {
         insert(&result, l1->data);
```

```
            l1 = l1->next;
            l2 = l2->next;
        }
    }
    return result;
}
int main()
{
    struct Node *L1 = NULL;
    struct Node *L2 = NULL;
    struct Node *Union = NULL;
    struct Node *Intersection = NULL;
    int n1, n2, value;
    printf("Enter number of elements in List 1: ");
    scanf("%d", &n1);
    printf("Enter %d sorted elements:\n", n1);
    for(int i = 0; i < n1; i++)
    {
        scanf("%d", &value);
        insert(&L1, value);
    }
    printf("Enter number of elements in List 2: ");
    scanf("%d", &n2);
    printf("Enter %d sorted elements:\n", n2);
    for(int i = 0; i < n2; i++)
    {
        scanf("%d", &value);
        insert(&L2, value);
    }
    printf("\nList 1:\n");
    display(L1);
    printf("\nList 2:\n");
    display(L2);
    Union = unionList(L1, L2);
    Intersection = intersectionList(L1, L2);
    printf("\nUnion of L1 and L2:\n");
    display(Union);
    printf("\nIntersection of L1 and L2:\n");
    display(Intersection);
    return 0;
}
```

Output

```
Enter number of elements in List 1: 4
Enter 4 sorted elements:
10 20 30 40
Enter number of elements in List 2: 4
Enter 4 sorted elements:
```

20 30 50 60

List 1:
10 -> 20 -> 30 -> 40 -> NULL
List 2:
20 -> 30 -> 50 -> 60 -> NULL
Union of L1 and L2:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL
Intersection of L1 and L2:
20 -> 30 -> NULL

2 . List 1:
1 3 5 7 9
List 2:
3 4 5 8 9

Union:
1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> NULL
Intersection:
3 -> 5 -> 9 -> NULL

3.Swap two elements by adjusting only the pointers (and not the data) using doubly linked list.

Code

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
// Create new node
struct Node* createNode(int data)
{
   struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}
// Insert at end
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
```

```
        return;
    }
    struct Node *temp = *head;
    while(temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}
// Display DLL
void display(struct Node *head)
{
    while(head != NULL)
    {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
// Swap two nodes by value
```

Output

```
Enter number of nodes: 5
Enter node values:
10 20 30 40 50
Enter first element to swap: 20
Enter second element to swap: 40
Original Doubly Linked List:
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL
Doubly Linked List after swapping:
10 <-> 40 <-> 30 <-> 20 <-> 50 <-> NULL
```

```
2. List:
1 2 3 4 5
Swap:
1 5
Original Doubly Linked List:
1 <-> 2 <-> 3 <-> 4 <-> 5 <-> NULL
Doubly Linked List after swapping:
    4    <-> 2 <-> 3 <-> 4 <-> 1 <-> NULL
```

4.Write an array implementation of self-adjusting lists. A self-adjusting list is like a regular list, except that all insertions are performed at the front of the list, and when an element is accessed by a find, it is moved to the front of the list, without changing the relative order of the other elements.

```
#include <stdio.h>

#define MAX 100
int list[MAX];
int n = 0;
// Insert at front
void insertFront(int value)
{
   if(n == MAX)
   {
      printf("List is Full!\n");
      return;
   }
   for(int i = n; i > 0; i--)
   {
      list[i] = list[i - 1];
   }
   list[0] = value;
   n++;
}
// Find and move to front
void findAndMoveToFront(int key)
{
   int pos = -1;
   for(int i = 0; i < n; i++)
   {
      if(list[i] == key)
      {
         pos = i;
         break;
      }
   }
   if(pos == -1)
   {
      printf("Element %d not found.\n", key);
      return;
   }
   int temp = list[pos];
   for(int i = pos; i > 0; i--)
   {
      list[i] = list[i - 1];
   }
   list[0] = temp;
   printf("Element %d found and moved to front.\n", key);
}
// Display list
```

```
void display()
{
   if(n == 0)
   {
      printf("List is Empty.\n");
      return;
   }
   printf("Self Adjusting List:\n");
   for(int i = 0; i < n; i++)
   {
      printf("%d ", list[i]);
   }
   printf("\n");
}
int main()
{
   int choice, value;
   while(1)
   {
      printf("\n----- SELF ADJUSTING LIST -----\n");
      printf("1. Insert at Front\n");
      printf("2. Find an Element\n");
      printf("3. Display List\n");
      printf("4. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);
      switch(choice)
      {
         case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            insertFront(value);
            break;
         case 2:
            printf("Enter element to find: ");
            scanf("%d", &value);
            findAndMoveToFront(value);
            break;
         case 3:
            display();
            break;
         case 4:
            return 0;
         default:
            printf("Invalid Choice!\n");
      }
   }
   return 0;
}
```

Output

```
Enter your choice: 1
Enter value to insert: 10
Enter your choice: 1
Enter value to insert: 20
Enter your choice: 1
Enter value to insert: 30
Enter your choice: 1
Enter value to insert: 40
Enter your choice: 3
Self Adjusting List:
40 30 20 10
Enter your choice: 2
Enter element to find: 20
Element 20 found and moved to front.
Enter your choice: 3
Self Adjusting List:
20 40 30 10
Enter your choice: 2
Enter element to find: 10
Element 10 found and moved to front.
Self Adjusting List:
    10   0 40 30
```

5.Write a C program to reverse a given doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
// Structure of Doubly Linked List Node
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
// Create a new node
struct Node* createNode(int data)
{
   struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}
// Insert node at end
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
   {
      temp = temp->next;
   }
   temp->next = newNode;
   newNode->prev = temp;
}
// Display Doubly Linked List
void display(struct Node *head)
{
   struct Node *temp = head;
   while(temp != NULL)
   {
      printf("%d <-> ", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");
}
// Reverse Doubly Linked List
```

```
void reverse(struct Node **head)
{
   struct Node *temp = NULL;
   struct Node *current = *head;
   while(current != NULL)
   {
      // Swap next and prev
      temp = current->prev;
      current->prev = current->next;
      current->next = temp;
      current = current->prev;
   }
   // Update head
   if(temp != NULL)
   {
      *head = temp->prev;
   }
}
int main()
{
   struct Node *head = NULL;
   int n, value;
   printf("Enter number of nodes: ");
   scanf("%d", &n);
   printf("Enter node values:\n");
   for(int i = 0; i < n; i++)
   {
      scanf("%d", &value);
      insert(&head, value);
   }
   printf("\nOriginal Doubly Linked List:\n");
   display(head);
   reverse(&head);
   printf("\nReversed Doubly Linked List:\n");
   display(head);
   return 0;
}
```

Output

```
Enter number of nodes: 5
Enter node values:
10 20 30 40 50
Original Doubly Linked List:
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL
Reversed Doubly Linked List:
50 <-> 40 <-> 30 <-> 20 <-> 10 <-> NULL
Enter number of nodes: 4
Enter node values:
1 2 3 4
Original Doubly Linked List:
```

1 <-> 2 <-> 3 <-> 4 <-> NULL
Reversed Doubly Linked List:
5 <-> 3 <-> 2 <-> 1 <-> NULL

6.Write a C program to delete the middle element of a DLL.

```
#include <stdio.h>
#include <stdlib.h>
// Structure for DLL Node
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
// Create new node
struct Node* createNode(int data)
{
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}
// Insert at end
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
   newNode->prev = temp;
}
// Display DLL
void display(struct Node *head)
{
   while(head != NULL)
   {
      printf("%d <-> ", head->data);
      head = head->next;
   }
   printf("NULL\n");
}
// Delete middle
```

Output

```
Enter number of nodes: 5
Enter node values:
```

10 20 30 40 50
Original Doubly Linked List:
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL
Middle node deleted successfully.
Doubly Linked List after deleting middle node:
10 <-> 20 <-> 40 <-> 50 <-> NULL
Enter number of nodes: 6
Enter node values:
5 10 15 20 25 30
Original Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 25 <-> 30 <-> NULL
Middle node deleted successfully.
Doubly Linked List after deleting middle node:
5 <-> 10 <-> 15 <-> 25 <-> 30 <-> NULL

7. Remove duplicate elements from a

(i) sorted DLL

(ii) unsorted DLL

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
struct Node* createNode(int data)
{
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
   newNode->prev = temp;
}
```

```
void display(struct Node *head)
{
   while(head != NULL)
   {
      printf("%d <-> ", head->data);
      head = head->next;
   }
   printf("NULL\n");
}
void removeDuplicates
```

Output

```
Enter number of nodes: 8
Enter sorted elements:
10 10 20 20 20 30 40 40
Original DLL:
10 <-> 10 <-> 20 <-> 20 <-> 20 <-> 30 <-> 40 <-> 40 <-> NULL
DLL after removing duplicates:
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

(ii) Remove Duplicate Elements from an Unsorted Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
struct Node* createNode(int data)
{
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;
```

```
    return newNode;
}
void insert(struct Node **head, int data)
{
    struct Node *newNode = createNode(data);
    if(*head == NULL)
    {
        *head = newNode;
        return;
    }
    struct Node *temp = *head;
    while(temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}
void display(struct Node *head)
{
    while(head != NULL)
    {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
void removeDuplicates(struct Node *head)
{
    struct Node *ptr1, *ptr2, *duplicate;
    ptr1 = head;
    while(ptr1 != NULL)
    {
        ptr2 = ptr1->next;
        while(ptr2 != NULL)
```

```
        {
            if(ptr1->data == ptr2->data)
            {
                duplicate = ptr2;
                ptr2 = ptr2->next;
                if(duplicate->prev)
                    duplicate->prev->next = duplicate->next;
                if(duplicate->next)
                    duplicate->next->prev = duplicate->prev;
                free(duplicate);
            }
            else
            {
                ptr2 = ptr2->next;
            }
        }
        ptr1 = ptr1->next;
    }
}
int main()
{
    struct Node *head = NULL;
    int n, value;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &value);
        insert(&head, value);
    }
    printf("\nOriginal DLL:\n");
    display(head);
```

```
    removeDuplicates(head);
    printf("\nDLL after removing duplicates:\n");
    display(head);
    return 0;
}
```

Output

```
Enter number of nodes: 8
Enter elements:
10 30 20 10 40 30 50 20
Original DLL:
10 <-> 30 <-> 20 <-> 10 <-> 40 <-> 30 <-> 50 <-> 20 <-> NULL
DLL after removing duplicates:
    10   -> 30 <-> 20 <-> 40 <-> 50 <-> NULL
```

7.Calculate the frequency count of a given integer in a circular DLL.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *prev;
   struct Node *next;
};
struct Node *head = NULL;
// Create Circular DLL
void createList(int n)
{
   struct Node *newNode, *temp = NULL;
   for(int i = 1; i <= n; i++)
   {
      newNode = (struct Node *)malloc(sizeof(struct Node));
      printf("Enter data for node %d: ", i);
      scanf("%d", &newNode->data);
      if(head == NULL)
      {
         head = newNode;
         head->next = head;
         head->prev = head;
         temp = head;
      }
      else
      {
         newNode->prev = temp;
         newNode->next = head;
         temp->next = newNode;
         head->prev = newNode;
         temp = newNode;
      }
   }
}
// Display Circular DLL
void display()
{
   if(head == NULL)
   {
      printf("List is empty.\n");
      return;
   }
   struct Node *temp = head;
   printf("\nCircular Doubly Linked List:\n");
   do
   {
      printf("%d <-> ", temp->data);
```

```
        temp = temp->next;
    }
    while(temp != head);
    printf("(HEAD)\n");
}
// Find Frequency
int frequencyCount(int key)
{
    if(head == NULL)
        return 0;
    int count = 0;
    struct Node *temp = head;
    do
    {
        if(temp->data == key)
            count++;
        temp = temp->next;
    }
    while(temp != head);
    return count;
}
int main()
{
    int n, key;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    createList(n);
    display();
    printf("\nEnter the integer to find frequency: ");
    scanf("%d", &key);
    int freq = frequencyCount(key);
    printf("\nFrequency of %d = %d\n", key, freq);
    return 0;
}
```

Output

```
Enter number of nodes: 7
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 10
Enter data for node 4: 30
Enter data for node 5: 10
Enter data for node 6: 40
Enter data for node 7: 20
Enter the integer to find frequency: 10
Circular Doubly Linked List:
10 <-> 20 <-> 10 <-> 30 <-> 10 <-> 40 <-> 20 <-> (HEAD)
Frequency of 10 = 3
List:
```

5 15 25 15 35 15
Key = 15
Circular Doubly Linked List:
5 <-> 15 <-> 25 <-> 15 <-> 35 <-> 15 <-> (HEAD)
Frequency of 15 = 3

9. Search for a particular element in a binary tree.

```
#include <stdio.h>
#include <stdlib.h>
// Structure of Binary Tree Node
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
// Create a new node
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node *)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
// Create Binary Tree
struct Node* createTree()
{
   int value;
   printf("Enter node value (-1 for no node): ");
   scanf("%d", &value);
   if(value == -1)
      return NULL;
   struct Node *root = createNode(value);
   printf("Enter left child of %d\n", value);
   root->left = createTree();
   printf("Enter right child of %d\n", value);
   root->right = createTree();
```

```
    return root;
}
// Inorder Traversal
void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
// Search Element
int search(struct Node *root, int key)
{
    if(root == NULL)
        return 0;
    if(root->data == key)
        return 1;
    return search(root->left, key) ||
           search(root->right, key);
}
int main()
{
    struct Node *root;
    int key;
    printf("Create Binary Tree\n");
    printf("Use -1 to indicate no node\n\n");
    root = createTree();
    printf("\nInorder Traversal of Tree:\n");
    inorder(root);
    printf("\n\nEnter element to search: ");
    scanf("%d", &key);
```

```
    if(search(root, key))
        printf("Element %d FOUND in the Binary Tree.\n", key);
    else
        printf("Element %d NOT FOUND in the Binary Tree.\n", key);
    return 0;
}
```

Output

```
Create Binary Tree
Use -1 to indicate no node
Enter node value (-1 for no node): 10
Enter left child of 10
Enter node value (-1 for no node): 20
Enter left child of 20
Enter node value (-1 for no node): -1
Enter right child of 20
Enter node value (-1 for no node): -1
Enter right child of 10
Enter node value (-1 for no node): 30
Enter left child of 30
Enter node value (-1 for no node): -1
Enter right child of 30
Enter node value (-1 for no node): -1
Enter element to search: 30
Inorder Traversal of Tree:
20 10 30
Enter element to search: 30
Element 30 FOUND in the Binary Tree.
Search Element: 40
Element 40 NOT FOUND in the Binary Tree.
```

30 → Found

40 → Not Found

10. Calculate the height or depth of a tree and print it.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
// Create a new node
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* createTree()
{
   int value;
   printf("Enter node value (-1 for no node): ");
   scanf("%d", &value);
   if(value == -1)
      return NULL;
   struct Node *root = createNode(value);
   printf("Enter left child of %d\n", value);
   root->left = createTree();
   printf("Enter right child of %d\n", value);
   root->right = createTree();
   return root;
}
```

```
void inorder(struct Node *root)
{
   if(root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
int height(struct Node *root)
{
   if(root == NULL)
      return 0;
   int leftHeight = height(root->left);
   int rightHeight = height(root->right);
   if(leftHeight > rightHeight)
      return leftHeight + 1;
   else
      return rightHeight + 1;
}
int main()
{
   struct Node *root;
   printf("Create Binary Tree\n");
   printf("Use -1 for NULL node\n\n");
   root = createTree();
   printf("\nInorder Traversal:\n");
   inorder(root);
   int h = height(root);
   printf("\n\nHeight (Depth) of the Tree = %d\n", h);
   return 0;
}
```

Output

Create Binary Tree

Use -1 for NULL node

Enter node value (-1 for no node): 10

Enter left child of 10

Enter node value (-1 for no node): 20

Enter left child of 20

Enter node value (-1 for no node): 40

Enter left child of 40

Enter node value (-1 for no node): -1

Enter right child of 40

Enter node value (-1 for no node): -1

Enter right child of 20

Enter node value (-1 for no node): -1

Enter right child of 10

Enter node value (-1 for no node): 30

Enter left child of 30

Enter node value (-1 for no node): -1

Enter right child of 30

Enter node value (-1 for no node): -1

Inorder Traversal:

40 20 10 30

Height (Depth) of the Tree = 3

Height (Depth) of the Tree = 4

11. Given a binary tree, convert this to a doubly linked list keeping the same order (Inorder traversal).

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node *prev = NULL;
struct Node *head = NULL;
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node *)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* createTree()
{
   int value;
   printf("Enter node value (-1 for NULL): ");
   scanf("%d", &value);
   if(value == -1)
      return NULL;
   struct Node *root = createNode(value);
   printf("Enter left child of %d\n", value);
   root->left = createTree();
   printf("Enter right child of %d\n", value);
   root->right = createTree();
   return root;
```

```
}
void inorder(struct Node *root)
{
   if(root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
void convertToDLL(struct Node *root)
{
   if(root == NULL)
      return;
   convertToDLL(root->left);
   if(prev == NULL)
   {
      head = root;
   }
   else
   {
      prev->right = root;
      root->left = prev;
   }
   prev = root;
   convertToDLL(root->right);
}
void displayDLL()
{
   struct Node *temp = head;
   printf("\nDoubly Linked List:\n");
   while(temp != NULL)
   {
```

```
        printf("%d", temp->data);
        if(temp->right != NULL)
            printf(" <-> ");
        temp = temp->right;
    }
    printf("\n");
}
int main()
{
    struct Node *root;
    printf("Create Binary Tree\n");
    printf("Use -1 for NULL node\n\n");
    root = createTree();
    printf("\nInorder Traversal of Binary Tree:\n");
    inorder(root);
    convertToDLL(root);
    printf("\n");
    displayDLL();
    return 0;
}
```

Output

```
Create Binary Tree
Use -1 for NULL node
Enter node value (-1 for NULL): 10
Enter left child of 10
Enter node value (-1 for NULL): 5
Enter left child of 5
Enter node value (-1 for NULL): 3
Enter left child of 3
Enter node value (-1 for NULL): -1
Enter right child of 3
Enter node value (-1 for NULL): -1
Enter right child of 5
```

Enter node value (-1 for NULL): 7

Enter left child of 7

Enter node value (-1 for NULL): -1

Enter right child of 7

Enter node value (-1 for NULL): -1

Enter right child of 10

Enter node value (-1 for NULL): 20

Enter left child of 20

Enter node value (-1 for NULL): 15

Enter left child of 15

Enter node value (-1 for NULL): -1

Enter right child of 15

Enter node value (-1 for NULL): -1

Enter right child of 20

Enter node value (-1 for NULL): 25

Enter left child of 25

Enter node value (-1 for NULL): -1

Enter right child of 25

Enter node value (-1 for NULL): -1

Inorder Traversal of Binary Tree:

3 5 7 10 15 20 25

Doubly Linked List:

3 <-> 5 <-> 7 <-> 10 <-> 15 <-> 20 <-> 25

12. Given a binary tree, the task is to connect the nodes that are at the same level using next right pointers.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
   struct Node *nextRight;
};
struct Node *queue[MAX];
int front = -1, rear = -1;
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node *)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   newNode->nextRight = NULL;
   return newNode;
}
void enqueue(struct Node *node)
{
   if(front == -1)
      front = 0;
   queue[++rear] = node;
}
struct Node* dequeue()
{
   return queue[front++];
}
```

```
int isEmpty()
{
   if(front > rear || front == -1)
      return 1;
   return 0;
}
struct Node* createTree()
{
   int value;
   printf("Enter node value (-1 for NULL): ");
   scanf("%d", &value);
   if(value == -1)
      return NULL;
   struct Node *root = createNode(value);
   printf("Enter left child of %d\n", value);
   root->left = createTree();
   printf("Enter right child of %d\n", value);
   root->right = createTree();
   return root;
}
void connectNodes(struct Node *root)
{
   if(root == NULL)
      return;
   front = rear = -1;
   enqueue(root);
   while(!isEmpty())
   {
      int levelSize = rear - front + 1;
      struct Node *prev = NULL;
      for(int i = 0; i < levelSize; i++)
      {
         struct Node *current = dequeue();
```

```
            if(prev != NULL)
                prev->nextRight = current;
            prev = current;
            if(current->left)
                enqueue(current->left);
            if(current->right)
                enqueue(current->right);
        }
        prev->nextRight = NULL;
    }
}
void printLevels(struct Node *root)
{
    if(root == NULL)
        return;
    front = rear = -1;
    enqueue(root);
    printf("\nNodes connected using nextRight pointer:\n\n");
    while(!isEmpty())
    {
        int levelSize = rear - front + 1;
        for(int i = 0; i < levelSize; i++)
        {
            struct Node *current = dequeue();
            printf("%d", current->data);
            if(current->nextRight)
                printf(" -> ");
            else
                printf(" -> NULL");
            if(current->left)
                enqueue(current->left);
            if(current->right)
                enqueue(current->right);
```

```
        }
        printf("\n");
    }
}
int main()
{
    struct Node *root;
    printf("Create Binary Tree\n");
    printf("Use -1 for NULL node\n\n");
    root = createTree();
    connectNodes(root);
    printLevels(root);
    return 0;
}
```

Output

```
Create Binary Tree
Use -1 for NULL node
Enter node value (-1 for NULL): 1
Enter left child of 1
Enter node value (-1 for NULL): 2
Enter left child of 2
Enter node value (-1 for NULL): 4
Enter left child of 4
Enter node value (-1 for NULL): -1
Enter right child of 4
Enter node value (-1 for NULL): -1
Enter right child of 2
Enter node value (-1 for NULL): 5
Enter left child of 5
Enter node value (-1 for NULL): -1
Enter right child of 5
Enter node value (-1 for NULL): -1
Enter right child of 1
```

Enter node value (-1 for NULL): 3

Enter left child of 3

Enter node value (-1 for NULL): 6

Enter left child of 6

Enter node value (-1 for NULL): -1

Enter right child of 6

Enter node value (-1 for NULL): -1

Enter right child of 3

Enter node value (-1 for NULL): 7

Enter left child of 7

Enter node value (-1 for NULL): -1

Enter right child of 7

Enter node value (-1 for NULL): -1

Nodes connected using nextRight pointer:

1 -> NULL

2 -> 3 -> NULL

4 -> 5 -> 6 -> 7 -> NULL

Level 0:

1 -> NULL

Level 1:

2 -> 3 -> NULL

Level 2:

4 -> 5 -> 6 -> 7 -> NULL

13. Implement priority queue using binary search tree and perform operations: Insert, Delete, Search.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* insert(struct Node *root, int data)
{
   if(root == NULL)
      return createNode(data);
   if(data < root->data)
      root->left = insert(root->left, data);
   else if(data > root->data)
      root->right = insert(root->right, data);
   return root;
}
struct Node* search(struct Node *root, int key)
{
   if(root == NULL || root->data == key)
      return root;
   if(key < root->data)
```

```
        return search(root->left, key);
    return search(root->right, key);
}
struct Node* findMin(struct Node *root)
{
    while(root->left != NULL)
        root = root->left;
    return root;
}
struct Node* deleteNode(struct Node *root, int key)
{
    if(root == NULL)
        return root;
    if(key < root->data)
        root->left = deleteNode(root->left, key);
    else if(key > root->data)
        root->right = deleteNode(root->right, key);
    else
    {
        if(root->left == NULL)
        {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }
        else if(root->right == NULL)
        {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }
        struct Node *temp = findMin(root->right);
        root->data = temp->data;
```

```
        root->right = deleteNode(root->right,
                                 temp->data);
    }
    return root;
}
void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
void highestPriority(struct Node *root)
{
    if(root == NULL)
    {
        printf("Priority Queue is Empty.\n");
        return;
    }
    struct Node *temp = root;
    while(temp->left != NULL)
        temp = temp->left;
    printf("Highest Priority Element = %d\n",
           temp->data);
}
int main()
{
    struct Node *root = NULL;
    int choice, value;
    while(1)
    {
```

```
printf("\n----- PRIORITY QUEUE USING BST -----\n");
printf("1. Insert\n");
printf("2. Delete\n");
printf("3. Search\n");
printf("4. Display\n");
printf("5. Show Highest Priority Element\n");
printf("6. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
   case 1:
      printf("Enter value to insert: ");
      scanf("%d", &value);
      root = insert(root, value);
      printf("%d inserted successfully.\n",
           value);
      break;
   case 2:
      printf("Enter value to delete: ");
      scanf("%d", &value);
      root = deleteNode(root, value);
      printf("%d deleted if present.\n",
           value);
      break;
   case 3:
      printf("Enter value to search: ");
      scanf("%d", &value);
      if(search(root, value))
         printf("%d found in Priority Queue.\n",
              value);
      else
         printf("%d not found.\n",
```

```
                value);
            break;
        case 4:
            printf("\nPriority Queue Elements:\n");
            inorder(root);
            printf("\n");
            break;
        case 5:
            highestPriority(root);
            break;
        case 6:
            exit(0);
        default:
            printf("Invalid choice!\n");
      }
   }
   return 0;
}
```

Output

```
Enter your choice: 1
Enter value to insert: 50
Enter your choice: 1
Enter value to insert: 20
Enter your choice: 1
Enter value to insert: 70
Enter your choice: 1
Enter value to insert: 10
Enter your choice: 4
Priority Queue Elements:
10 20 50 70
Enter your choice: 5
Highest Priority Element = 10
Enter your choice: 3
```

Enter value to search: 50

50 found in Priority Queue.

Enter your choice: 2

Enter value to delete: 20

Enter your choice: 4

Priority Queue Elements:

10 50 70

14. In a given binary search tree, write routines to delete a node with no children, one child, and two children.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* insert(struct Node *root, int data)
{
   if(root == NULL)
      return createNode(data);
   if(data < root->data)
      root->left = insert(root->left, data);
   else if(data > root->data)
      root->right = insert(root->right, data);
   return root;
}
struct Node* findMin(struct Node *root)
{
   while(root->left != NULL)
      root = root->left;
   return root;
```

```
}
struct Node* deleteNode(struct Node *root, int key)
{
   if(root == NULL)
      return root;
   // Search the node
   if(key < root->data)
      root->left = deleteNode(root->left, key);
   else if(key > root->data)
      root->right = deleteNode(root->right, key);
   else
   {
      // Case 1: No Child
      if(root->left == NULL && root->right == NULL)
      {
         free(root);
         return NULL;
      }
      // Case 2: One Child
      else if(root->left == NULL)
      {
         struct Node *temp = root->right;
         free(root);
         return
```

Output

```
Enter number of nodes: 7
Enter BST elements:
50 30 70 20 40 60 80
Enter node to delete: 20
BST (Inorder Traversal):
20 30 40 50 60 70 80
BST after deletion:
30 40 50 60 70 80
```

Enter node to delete: 30

BST after deletion:

40 50

Enter node to delete: 50

BST after deletion:

20 30 40 60 70 80

15. Convert the given postfix expression to an expression tree.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX 100
// Expression Tree Node
struct Node
{
   char data;
   struct Node *left;
   struct Node *right;
};
struct Node* stack[MAX];
int top = -1;
struct Node* createNode(char data)
{
   struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
void push(struct Node* node)
{
   stack[++top] = node;
}
struct Node* pop()
{
   return stack[top--];
}
struct Node* constructTree(char postfix[])
{
```

```
    int i;
    for(i = 0; postfix[i] != '\0'; i++)
    {
        char ch = postfix[i];
        if(isalnum(ch))
        {
            push(createNode(ch));
        }
        else
        {
            struct Node* opNode = createNode(ch);
            opNode->right = pop();
            opNode->left = pop();
            push(opNode);
        }
```

```
 Enter a postfix expression: ab-cd*+
(a - b) + (c * d)
Inorder Traversal:
a - b + c * d
Preorder Traversal:
+ - a b * c d
Postorder Traversal:
a b - c d * +
```

16. Construct binary search tree representation using:

(i) Inorder node

(ii) Preorder and Postorder

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* insert(struct Node *root, int data)
{
   if(root == NULL)
      return createNode(data);
   if(data < root->data)
      root->left = insert(root->left, data);
   else
      root->right = insert(root->right, data);
   return root;
}
void inorder(struct Node *root)
{
   if(root)
```

```
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
void preorder(struct Node *root)
{
    if(root)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
int main()
{
    struct Node *root = NULL;
    int n, value;
    printf("Enter number of nodes in Preorder Traversal: ");
    scanf("%d", &n);
    printf("Enter Preorder elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &value);
        root = insert(root, value);
    }
    printf("\nConstructed BST Inorder Traversal:\n");
    inorder(root);
    printf("\nConstructed BST Preorder Traversal:\n");
    preorder(root);
    printf("\n");
    return 0;
```

```
}
```

Output

```
Enter number of nodes in Preorder Traversal: 6
Enter Preorder elements:
50 30 20 40 70 60
Constructed BST Inorder Traversal:
20 30 40 50 60 70
Constructed BST Preorder Traversal:
50 30 20 40 70 60
```

(ii) Construct BST from Postorder Traversal

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
struct Node* insert(struct Node *root, int data)
{
   if(root == NULL)
      return createNode(data);
   if(data < root->data)
      root->left = insert(root->left, data);
```

```
    else
        root->right = insert(root->right, data);
    return root;
}
void inorder(struct Node *root)
{
    if(root)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
void postorder(struct Node *root)
{
    if(root)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
int main()
{
    struct Node *root = NULL;
    int n;
    printf("Enter number of nodes in Postorder Traversal: ");
    scanf("%d", &n);
    int post[n];
    printf("Enter Postorder elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &post[i]);
```

```
    }
    for(int i = n - 1; i >= 0; i--)
    {
        root = insert(root, post[i]);
    }
    printf("\nConstructed BST Inorder Traversal:\n");
    inorder(root);
    printf("\nConstructed BST Postorder Traversal:\n");
    postorder(root);
    printf("\n");
    return 0;
}
Enter number of nodes in Postorder Traversal: 6
Enter Postorder elements:
20 40 30 60 70 50
Constructed BST Inorder Traversal:
20 30 40 50 60 70
Constructed BST Postorder Traversal:
20 40 30 60 70 50
```

17. Implement heap sort for a set of numbers:

I/p: [50, 20, 30, 5, 15, 22, 32]

Heap: [50, 25, 32, 20, 15, 22, 30] n = 7

```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   if(left < n && arr[left] > arr[largest])
      largest = left;
   if(right < n && arr[right] > arr[largest])
      largest = right;
   if(largest != i)
   {
      swap(&arr[i], &arr[largest]);
      heapify(arr, n, largest);
   }
}
void buildHeap(int arr[], int n)
{
   for(int i = n/2 - 1; i >= 0; i--)
   {
      heapify(arr, n, i);
   }
}
void heapSort(int arr[], int n)
```

```
{
    buildHeap(arr, n);
    printf("\nMax Heap:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    for(int i = n - 1; i > 0; i--)
    {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
void display(int arr[], int n)
{
    for(int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("\nOriginal Array:\n");
    display(arr, n);
```

```
    heapSort(arr, n);
    printf("\nSorted Array using Heap Sort:\n");
    display(arr, n);
    return 0;
}
```

Output

```
Enter number of elements: 7
Enter 7 elements:
50 20 30 5 15 22 32
Original Array:
50 20 30 5 15 22 32
Max Heap:
50 20 32 5 15 22 30
Sorted Array using Heap Sort:
5 15 20 22 30 32 50
50 20 30 5 15 22 32
```